

CSIE 5452, Spring 2026: Homework 2

B13902022 Yu-Chi,Lai

1 Simulated Annealing for Priority Assignment

```
1  #include <stdio>
2  #include <fstream>
3  #include <cmath>
4  #include <chrono>
5  #include <cfloat>
6  #include <ctime>
7  #define MAX_LENGTH 50
8  #define TIME_LIMIT 15000000
9  const double INF = 1e100;
10
11 typedef struct msg {
12     int p; // priority
13     double tt; // transmission time;
14     double T; // period
15 } Msg;
16
17 double wrct(int i, int n, double tau, Msg m[]);
18
19 void print_info(int n, double tau, Msg m[]) {
20     printf("%d, %f\n", n, tau);
21     for (int i = 0; i < n; i++) {
22         printf("(%d %f %f): %f\n", m[i].p, m[i].tt, m[i].T, wrct(i, n, tau, m));
23     }
24 }
25
26 double wrct(int i, int n, double tau, Msg m[]) { // worst case response time
27     double Ci = m[i].tt;
28     double Bi = 0.0;
29     for (int k = 0; k < n; k++) {
30         if (m[k].p >= m[i].p) {
31             Bi = std::max(Bi, m[k].tt);
32         }
33     }
34
35     double Qi = Bi;
36     while (1) {
37         double temp = 0.0;
38         for (int k = 0; k < n; k++) {
39             if (m[k].p < m[i].p) {
40                 temp += std::ceil((Qi+tau)/m[k].T)*m[k].tt;
41             }
```

```
42     }
43     temp += Bi;
44
45     if (temp > m[i].T) {
46         return -1.0; // not schedulable
47     }
48
49     if (std::fabs(Qi-temp) < 1e-9) {
50         return Qi + Ci;
51     } else {
52         Qi = temp;
53     }
54 }
55 }
56
57 double sum_wcrct(int n, double tau, Msg m[]) {
58     double sum = 0.0;
59     for (int i = 0; i < n; i++) {
60         double Ri = wcrct(i, n, tau, m);
61         if (Ri < 0) return INF;
62         sum += Ri;
63     }
64     return sum;
65 }
66
67 int main() {
68     auto beg = std::chrono::high_resolution_clock::now();
69     srand(time(NULL));
70
71     int n;
72     double tau;
73     Msg m[MAX_LENGTH];
74     int bestP[MAX_LENGTH]; // best priority assignment
75     double minsum = DBL_MAX;
76
77     std::ifstream fin("input.dat");
78     fin >> n;
79     fin >> tau;
80
81     for (int i = 0; i < n; i++) {
82         fin >> m[i].p >> m[i].tt >> m[i].T;
83         bestP[i] = m[i].p;
84     }
85
86     double curr = sum_wcrct(n, tau, m);
87     if (curr < minsum) {
88         minsum = curr;
89         for (int i = 0; i < n; i++) bestP[i] = m[i].p;
90     }
91
92     double temperature = 5.0;
```

```
93     const double temp_min = 1e-6;
94     const double alpha = 0.997;
95     const int loops = 300;
96
97     while (temperature > temp_min) {
98         auto end = std::chrono::high_resolution_clock::now();
99         auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - beg);
100         if (duration.count() >= TIME_LIMIT) {
101             printf("%d ms passed", duration.count());
102             break;
103         }
104
105         for (int i = 0; i < loops; i++) {
106             int a = rand() % n, b = rand() % n;
107             while (a == b) b = rand() % n;
108             std::swap(m[a].p, m[b].p);
109
110             double cand = sum_wcrtn(n, tau, m);
111             double delta = cand - curr;
112
113             bool accept = false;
114             if (delta <= 0.0) accept = true;
115             else {
116                 double prob = std::exp(-delta / temperature);
117                 double x = (double) rand() / (RAND_MAX + 1.0);
118                 if (x <= prob) accept = true;
119             }
120
121             if (accept) {
122                 curr = cand;
123                 if (cand < minsum) {
124                     minsum = cand;
125                     for (int i = 0; i < n; i++) bestP[i] = m[i].p;
126                 }
127             } else {
128                 std::swap(m[a].p, m[b].p);
129             }
130         }
131
132         temperature *= alpha;
133     }
134     //
135
136     for (int i = 0; i < n; i++) {
137         printf("%d\n", bestP[i]);
138     }
139     printf("Optimized result: %f\n", minsum);
140
141 }
```

```
→ hw2 git:(main) ✗ ./a.out
14
6
0
7
3
5
11
2
9
10
1
16
4
8
15
13
12
Optimized result: 194.520000
```

Figure 1: The optimized result obtained from SA

2 Intersection Management: Part I

1.

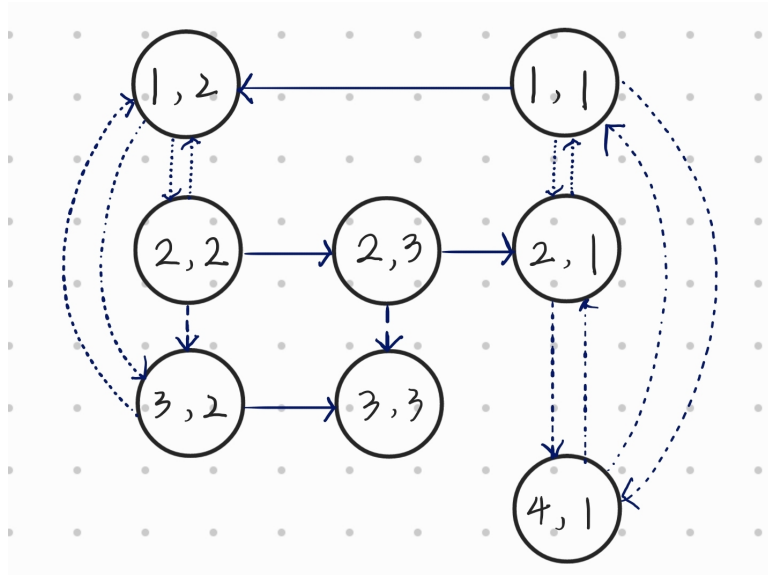


Figure 2: The timing conflict graph

2.

The following graph gives an acyclic solution containing DEADLOCK. The scheduling should be like:

1. vehicle 2 enter zone 2, then enter zone 3, then waits for zone 1.
2. vehicle 4 enter zone 1, then leaves.
3. vehicle 1 enters zone 1, then waits for zone 2.
4. vehicle 3 enters zone 2, then waits for zone 3.

which is trivially a case of DEADLOCK.

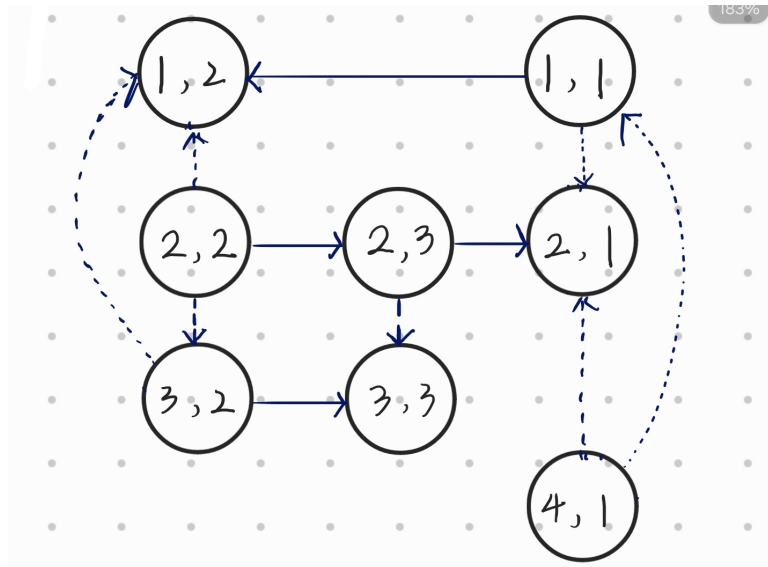


Figure 3: An example of deadlock

We can also construct the resource conflict graph, and we will notice that there exists one cycle. $((1,1,2) \rightarrow (2,3,1) \rightarrow (3,2,3) \rightarrow (1,1,2))$ Hence, there's a DEADLOCK.

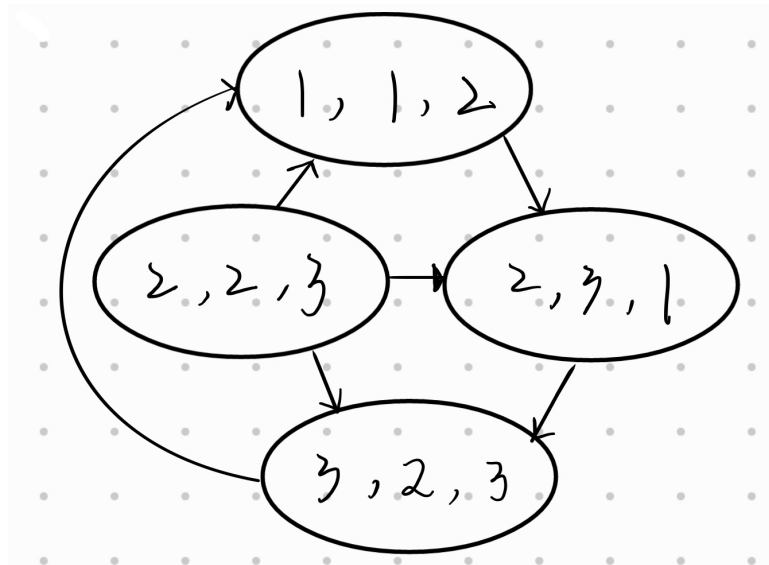


Figure 4: The resource conflict graph of this example

3 Intersection Management: Part II

When the entire intersection is modeled as one conflict zone, every vehicle i has exactly one node $(i, 1)$ in the timing conflict graph G . This has two consequences:

1. No Type-1 edges exist: Type-1 edges connect consecutive conflict zones of the same vehicle, but with only one zone, no vehicle has consecutive zones. So the only edges in G are Type-2 (same lane) and Type-3 (different lanes).
2. The resource conflict graph H' is always empty. All five construction rules (Rules 1-5) require at least one vehicle to have two consecutive conflict zones in order to generate an H' node of the form (i, j, j') . With a single conflict zone, this prerequisite is never met. Therefore H' has no nodes and no edges, it is trivially acyclic, and deadlock-freeness is automatically guaranteed.
3. Expressiveness is severely limited. Since any two vehicles from different lanes share the only conflict zone, the Type-3 edges in G' form a total ordering of all vehicles. Only one vehicle may be in the intersection at a time, regardless of whether their actual paths would physically conflict (e.g., a right turn vs. a straight-through would still be serialized unnecessarily).

We can use any total-ordering policy: FCFS, priority-based, etc. to assign a strict sequence to all vehicles. Because H' is always empty, no deadlock-verification step is needed at all. This is precisely what a stop sign implements: vehicles queue and take turns one at a time. The cost is minimal throughput; the benefit is trivial safety and correctness guarantees.

4 Intersection Management: Part III

The Conditions: The condition to simplify the rules is to guarantee that every vehicle involved in a conflict has a preceding zone and a succeeding zone relative to that conflict zone. In a system model, this condition can be universally satisfied by padding the beginning and end of every vehicle's trajectory with "dummy" (or virtual) source and sink zones. By doing this, every actual physical conflict zone inside the intersection is treated strictly as an internal node.

The Two Rules: Under this condition, we only need to apply **Rule 1** and **Rule 2** to construct the resource conflict graph H' from the timing conflict graph G .

- **Rule 1 (Continuous Movement):** This rule links a single vehicle's transitions together to ensure its path remains continuous. For a vehicle i traveling from zone j' to j , and then from j to j'' , we connect its arrival transition to its departure transition:

$$v_{i,j',j} \rightarrow v_{i,j,j''}$$

- **Rule 2 (Conflict Resolution):** This rule enforces the timing priority when two vehicles compete for the same resource. If vehicle i has priority over vehicle k at conflict zone j (represented in G as $v_{i,j} \dashrightarrow v_{k,j}$), we link their departure transitions. This strictly requires vehicle i to transition out of j before vehicle k is allowed to transition out of j :

$$v_{i,j,j''} \rightarrow v_{k,j,j^{**}}$$

The Benefits: By introducing the dummy zone condition, we convert all boundary scenarios into internal transitions, completely eliminating the need for **Rule 4** and **Rule 5** (which exist solely to handle the edge cases of vehicles spawning or terminating without prior or subsequent zones). Additionally, because Rule 2 and Rule 3 both enforce priority equivalently, we only need one of them to detect cycles. Using only two rules heavily simplifies the graph construction algorithm, eliminates messy conditional logic needed for boundary checking, and makes the overall deadlock-freeness verification much more computationally efficient.

Basically, we can realize rule 3, 4 and 5 using only rule 1 and 2 under our previously defined conditions.

5 Realization of Level-X Autonomy

The realization of autonomous driving is not merely a technological timeline, but a complex intersection of hardware costs, legal liability, regulatory frameworks, and human trust. Below is an estimation of when each level will be broadly realized, accompanied by real-world examples and multi-disciplinary justifications.

Level-3 Autonomy (Conditional Driving Automation)

Estimated Realization: Currently being realized in limited consumer markets (2024–2028).

Justification: From a *technological* standpoint, sensor fusion (camera, radar, LiDAR) is mature enough to handle structured environments like highways. However, the primary bottleneck has been *human comfort* and *law*. Level 3 introduces the dangerous “hand-over” problem, where a distracted human must suddenly take control if the system fails. Because the manufacturer assumes legal liability while the system is active, many automakers bypassed Level 3 entirely. Slowly, *regulatory* bodies are establishing frameworks to govern this liability.

Example: Mercedes-Benz’s *Drive Pilot* system. It is the first internationally certified Level-3 system, legally approved to operate in jurisdictions like Nevada and California. Under specific conditions (e.g., highway traffic jams under 40 mph), the driver is legally permitted to take their eyes off the road to watch a movie or read, but must remain physically ready to take the wheel if the system prompts them.

Level-4 Autonomy (High Driving Automation)

Estimated Realization: Currently realized in commercial fleets; widespread consumer ownership expected by 2030–2035.

Justification: Level 4 removes the human entirely, but only within a strict Operational Design Domain (ODD)—such as a geofenced city or specific weather conditions. *Technologically*, this is viable today by relying on ultra-high-definition pre-mapped routes. The main barrier is *cost*; the dense array of LiDAR and computing hardware required makes personal vehicle ownership prohibitively expensive. Therefore, the business model is currently restricted to commercial ridesharing. *Regulations* are also heavily localized, requiring city-by-city approval.

Example: *Waymo One* operates Level-4 fully autonomous robotaxis in cities like Phoenix, Los Angeles, and San Francisco. These vehicles operate with no human driver inside, but they are strictly bound by their ODD. If a Waymo vehicle encounters an unmapped construction zone or severe weather outside its programming, it will pull over and safely stop rather than ask for human help.

Level-5 Autonomy (Full Driving Automation)

Estimated Realization: Several decades away (2050 or beyond).

Justification: Level 5 implies that the vehicle can drive anywhere a human can, under any conditions, without an ODD. *Technologically*, this requires a leap from advanced machine learning to something resembling Artificial General Intelligence (AGI). The system must possess human-like reasoning to negotiate unwritten social rules of driving (e.g., making “eye contact” with a pedestrian, or interpreting a traffic cop’s hand gestures). The *legal* implications of a globally operating autonomous system and the *human trust* required to sleep in a car hurtling through a severe snowstorm make this a distant reality.

Example: No current system is close to this. A true Level-5 vehicle would need to seamlessly transition from navigating a perfectly marked highway in sunny California to safely traversing an unmapped, chaotic intersection in Mumbai during a monsoon, all without any remote assistance or prior mapping.